MAVEN

definition: Maven is a build automation and project management tool mainly used for java projects
(1) (2)
helps to compile code, manage dependencies, run tests and package applications.
(3)

.jar. Multiple .class files bundled together makes a .jar file.

Artifact: Artifact is a file produced by software build process
↳ contains ↳ compiled code
↳ required resources
needed to run or deploy the application.

Stored in AWS CodeArtifact, Artifactory, Jfrog, etc.

Source code → (build tool / Automates) → Artifact.

Notewise

dependencies

External libraries or packages that an application needs to work properly, provide readymade functionalities to work with instead of writing everything from scratch

deployment workflow: Dev → UAT → production

(testing of new features or code changes)

(testing requirements)

(approved & tested for customer use).

Build tool task: Automates :-

1. Downloading dependencies
2. Compiling Source code
3. packaging
4. testing
5. documentation
6. deployment.

(DCPTDD)

Egs. ANT, Maven, Gradle.

Notewise

Input file.

Input file for buildtool is a configuration file which provides necessary info about building the project to the build tool.

for maven input file is <pom.xml>

pom.xml

must be present in base directory of the buildtool default input file Maven to build our projects.

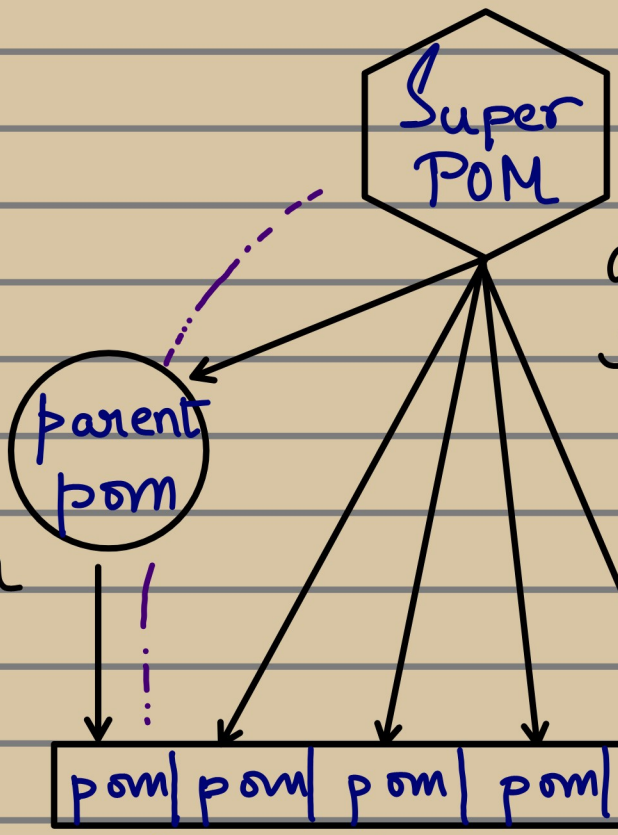
<Contents>

project root element
properties.
dependencies
plugin
goals
profile
version
mail list

<Contents>

Notewise

Types of pom



Contains generic configurations and information required by multiple builds. referenced by other poms.

pom inherited by another pom shares common dependencies config among multiple objects.

directly required by projects contains project specific details.

maven

Combines info from all to form a template

---

fact: Maven uses convention over configuration, which means developers are not required to create build process themselves

Notewise

Project Object Model.

fundamental unit of work in Maven.

Contains info about: project config used during build process.

Pom also contains goals and plugins. during executing task.

Pom.xml has:

- Project dependencies defined
- Plugins
- goals
- mailing list
- build profile
- project version

Min req:

project root
<model version>
<group ID>
<artifact_ID> } (GAV)
<version>

Notewise

Maven lifecycle.

(Clean → Build → Site)

clean. .java → .class → directory

recompile ↑ del old

Sequence of phases run in order.

phase — Step in build (0 or more goals bound to it)

bound

Goal — Actual task to be performed

plugin ⇝ Collection of goals.

maven compile

pom — Local.

maven → Search for plugin.

compile

Super pom → m-c-p { G, A, V }

plugin.

phases & Goals

compile. task

Notewise

Maven lifecycle:

Validate (check for correct Configuration)

↓

Compile (code → .class)

↓

test

↓

package appl. → JAR, WAR.

↓

Verify

↓

Install generated Artifact → .m2.

↓

deploy uploads Artifact to remote repo.

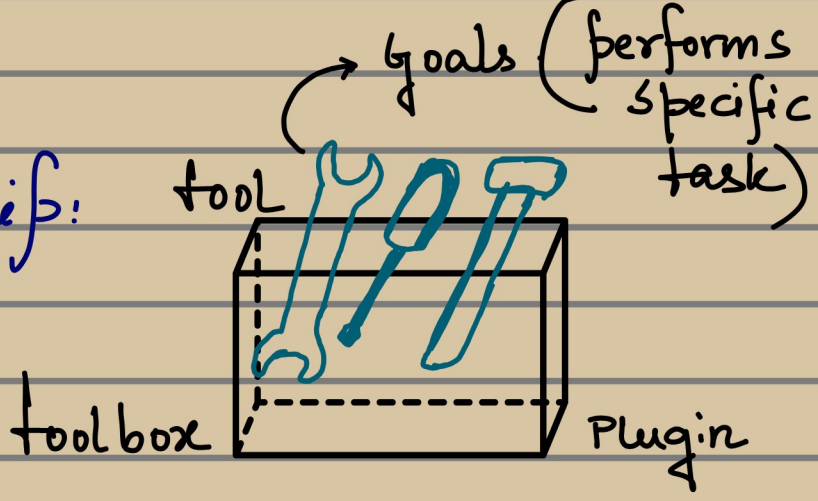
Notewise

Plugin & goal relation-ship

Plugin collection of tasks to be performed by Maven (compile, testing, packaging)

Goal however refers to any single task provided by plugin

Relationship:



hence we can say plugin = collection of goals.

Version: x.y.z → build (minor), → patch, → Generation (major)

Version in maven is referred as specific build / release of the project or dependency

helps maven determine correct ver of artifact to use

Notewise

Settings.xml — Maven Configuration file used to define settings that are not specific to a single project.

Can contain

- repository configuration
- credentials for private repository
- proxy settings
- profile and environment specific settings.

Pom.xml (details) — pom.xml <contents>

A) project root element.

contains important project related information, root element of every maven file.

Syntax / Content

<project>

- <modelVersion>

Notewise

- <groupID>
- <artifactId>
- <version>

- <dependencies>
- <build>

</project>

B) properties

used to declare / define custom variables values that can be reused throughout pom.xml.

avoid repeatation, promotes reusability

<properties>

<property-name> value </property-name>

</>

${value.name} ..... use any value.

Notewise

c) Dependencies

The <dependencies> element is used to define the external libraries that a Maven project needs.

Maven automatically downloads required dependencies from a configured repo and adds them to the project.

Syntax

```
<dependencies>
  <dependency>
      <groupId></>
      <artifact></>
      <version></>
  </>
</>
```

Transitive dependency: Transitive dependency is automatically included by Maven because direct dependency needs it.

Notewise

Eg. project need LibA → needs.
maven includes LibB

D) Plugins.

used to Configure Maven plugins that perform specific tasks during the build process.

Plugins can compile code, run tests, package application, generate report.

Plugin + Goal

Plugin → Contains goals → Goals perform action.

```
<build>
  <plugins>
    <plugin>
      G
      A
      V
      <config>
        <source>
        <target>
      </config>
    </plugin>
  </>
```

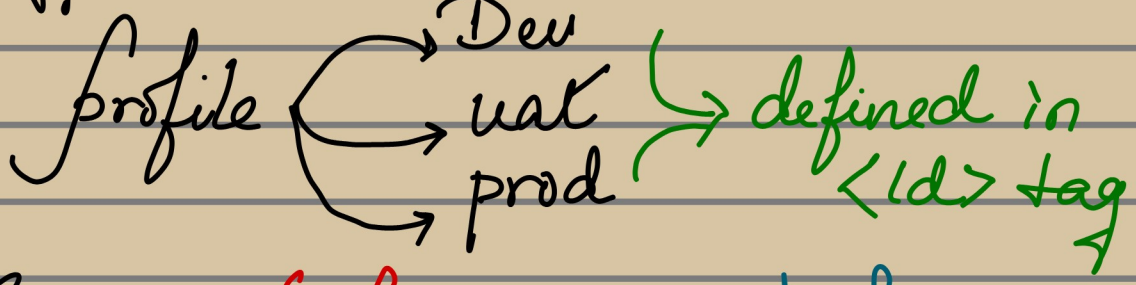
Notewise

G) Goals

Goals is a specific task or action performed by Maven plugin

mvn <plugin-prefix>:<goal>

H) Profile: Set of Configuration values that can be activated to customise the maven build for different environment or situation



Syntax:

```
<profiles>
  <profile>
    <id> p_name </id>
    <properties>
      values specific to that profile.
    </>
  </>
</>
```

profile name

Notewise

How to run profile: `mvn compile -Pdev`

`mvn compile -Pdev,P1,P2,P3.`

multi-profiling: Yes we can activate multiple profile by separating their ID's with commas.

`-Pdev,P2,P3 .....`

GAV: GroupId + ArtifactId + Version.

(Unique Maven Coord.) Combination Maven uses to uniquely identify a project in a maven repository

(owner) 1. GroupId: groupId identifies the organisation, company, or group that owns the project.

(unique identifier) 2. ArtifactId: identifies the actual project/library name.

3. Version: identifies the specific release of that artifact

Notewise

Eg. <groupId> com.example </>
<artifactId> payment-service </>
<version> 3.14.0 </>.

Case Study:

```
<dependency>
    <grpId>
        org.apache.commons.
    </>
    <artifactId>
        commons-lang3
    </>
    <version>
        3.14.0
    </>
</dependency>
```

Maven will compile it as:

org.apache.commons:commons-lang3:3.14.0.

Structure

Repository
- groupId
  - artifactId
    - version
      - artifact-file

Notewise

Sample → pom.xml

Here is a simple sample pom.xml showing the important sections:

```
<?xml version="1.0" encoding="UTF-8"?>

<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <!-- POM Model Version -->
    <modelVersion>4.0.0</modelVersion>

    <!-- GAV -->
    <groupId>com.example</groupId>
    <artifactId>my-app</artifactId>
    <version>1.0.0</version>

    <!-- Packaging -->
    <packaging>jar</packaging>

    <!-- Project Properties -->
    <properties>
        <maven.compiler.source>17</maven.compiler.source>
        <maven.compiler.target>17</maven.compiler.target>
    </properties>

    <!-- Dependencies -->
    <dependencies>

        <dependency>
            <groupId>org.junit.jupiter</groupId>
            <artifactId>junit-jupiter</artifactId>
            <version>5.10.0</version>
            <scope>test</scope>
        </dependency>

    </dependencies>

    <!-- Build Configuration -->
    <build>
        <plugins>

            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>
            </plugin>

        </plugins>
    </build>

</project>
```

Main parts to remember

Section	Purpose
<modelVersion>	Defines POM model version
<groupId>	Identifies the organization/project
<artifactId>	Name of the project/artifact
<version>	Project version
<packaging>	JAR, WAR, etc.
<properties>	Stores project configuration/properties
<dependencies>	Libraries required by the project
<build>	Build configuration
<plugins>	Tools that perform Maven build tasks

GAV = GroupId + ArtifactId + Version

```
com.example : my-app : 1.0.0
    ↑          ↑      ↑
 GroupId    ArtifactId Version
```

Notewise

Maven Repository.

Storage location where maven keeps artifacts, maven uses this repos to find dependencies needed by project to store/download build artifacts.

3 main types of

Local Repository | Central Repository | Remote Repository

Local repository

Stores dependencies/ artifacts on our own machine.

local repository is a folder on local machine where maven stores dependencies and artifact that it has downloaded or installed

Workflow:

Maven needs any artifact

checks.

~/.m2/repo

if not found

downloads it to .m2

Notewise

Default location : ~/.m2/repository

local repository is used for:

- Caching downloaded dependencies
- Avoiding repealed downloads
- Storing artifacts, plugins
- Making builds faster.

Central Repo: public repository over the INTERNET where most widely used dependencies, Java lib, maven artifacts are stored.

Used for:

- Downloading publicly available dependencies
- Getting open-source libraries

Remote repository: Remote repo stores custom-modified artifacts generally company specific often are

Notewise

private and not open to public (Internal Company Artifact)

Owned by company or Organisation.

Nexus / Jfrog / Artifactory.

Maven Archetype:

Instead of manually creating project template and directory

mvn archetype:generate

```
my-project/
  ↳ pom.xml
  ↳ src/
      ↳ main/
          ↳ java/
      ↳ test/
          ↳ java/
```

Notewise